Classical Machine Learning Models

KAUST–Mawhiba IOAI Training

Week 1 · Day 5

KAUST Academy
King Abdullah University of Science and Technology

April 25, 2026

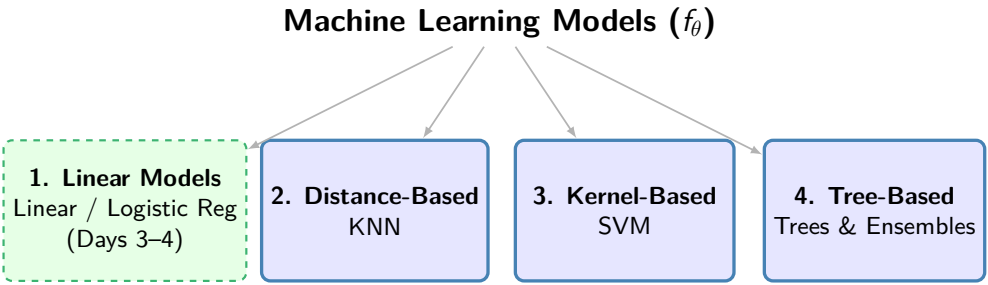
Table of Contents

Recap: Where We Are

Day 3: Built our first models (Linear & Logistic Regression).
Day 4: Learned why models fail (overfitting) and how to fix it (regularization, cross-validation).

Today we expand our toolkit beyond linear models.
"Apply cross-validation and regularization from Day 4 to compare these new models fairly."

The ML Toolbox: Types of Models

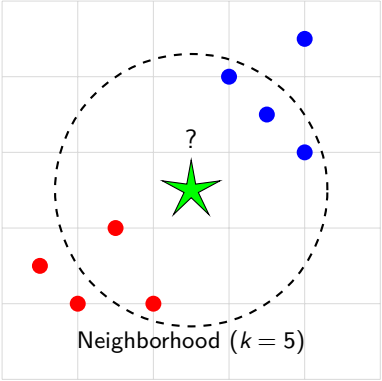


k-Nearest Neighbors (k-NN): Concept

Linear models learn a global formula ($\mathbf{w}^T\mathbf{x}$). **k-NN** is different: it is a **Lazy Learner**. It memorizes the training data and predicts based on local similarity — **no parameters** to learn.

The Algorithm:

1. Choose a distance metric $d(\cdot, \cdot)$ and an integer k.
2. For a new point x_{new}, find the k closest training points.
3. **Classification:** Majority vote.
4. **Regression:** Average of neighbors.



k-NN: Mathematics of Distance

Performance of k-NN depends heavily on how we define "closeness."

1. **Euclidean Distance (L_2 Norm):** Most common

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^{d} (x_i - y_i)^2} = ||\mathbf{x} - \mathbf{y}||_2$$

2. **Manhattan Distance (L_1 Norm):** Good for high dimensions

$$d(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^{d} |x_i - y_i| = ||\mathbf{x} - \mathbf{y}||_1$$

Important: Since distance depends on feature magnitude, **scaling is essential**! (Remember Day 2: StandardScaler / MinMaxScaler)

k-NN: Advantages and Disadvantages

Advantages

- **Simple:** Easy to understand and implement
- **No Training:** Training time is effectively zero ($O(1)$)
- **Non-Linear:** Naturally captures complex decision boundaries

Disadvantages

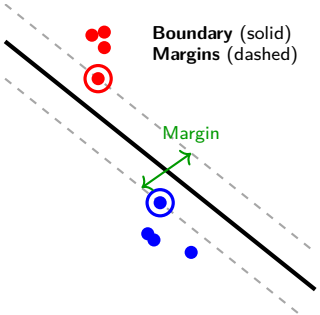
- **Slow Inference:** Prediction is $O(N)$ — must scan all data
- **Memory Intensive:** Must store the entire dataset
- **Scaling Required:** Sensitive to feature magnitude
- **Curse of Dimensionality:** Degrades in high dimensions

Hyperparameter: k controls complexity. Small $k \rightarrow$ overfitting. Large $k \rightarrow$ underfitting.

Support Vector Machines (SVM)

"Success isn't about barely crossing the line. It's about having some room to spare."

Logistic Regression learns a boundary that fits probabilities. SVM chooses the boundary with the **largest safety gap (Margin)**.



- **Hyperplane:** boundary ($\mathbf{w}^T\mathbf{x} + b = 0$)
- **Margin:** "safety gap" to the closest points
- **Support Vectors:** closest points that **define** the boundary

Bigger margin $\Rightarrow$ more robust to noise.

Hard Margin SVM

We want the **widest possible margin**. Maximizing width $\iff$ minimizing weights $||\mathbf{w}||$.

The "Hard Margin" Objective

$$\min_{\mathbf{w},b} \frac{1}{2}||\mathbf{w}||^2$$

Subject to: $y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1$

What does this mean?

- **Minimize Objective:** Keep the weights small
- **Subject to Constraint:** Strictly forbid ANY data point from being inside the margin

Assumes data is perfectly linearly separable.

Soft Margin: When Perfection is Impossible

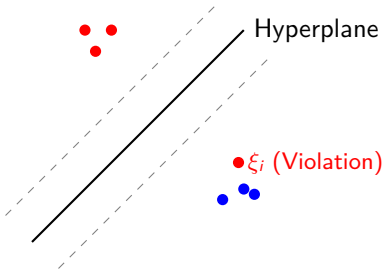
Real data is noisy. A Hard Margin would crash if just one outlier existed.

Solution: Allow some "violations" (ξ_i) but penalize them using hyperparameter **C**.

The Hyperparameter C

Large C (Strict)
- Penalizes errors heavily
- Narrow margin, risk of overfitting

Small C (Tolerant)
- Ignores small errors
- Wider margin, better generalization



Handling Non-Linearity: The Kernel Trick

Problem: Standard SVM finds a linear boundary. What if data is not linearly separable?

The Solution: Map inputs to a higher-dimensional space where they *become* separable.

$$\mathbf{x} \rightarrow \phi(\mathbf{x})$$

Issue: Computing $\phi(\mathbf{x})$ explicitly is expensive.

The "Trick": Use **Kernel Functions** that compute high-dimensional similarity **directly** without building ϕ:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$$

Common SVM Kernels

The choice of Kernel tells the SVM "how to measure similarity."

- **1. Linear Kernel** (Good for linear problems)
  - $K(\mathbf{x}, \mathbf{y}) = \mathbf{x}^T\mathbf{y}$
- **2. Polynomial Kernel**
  - $K(\mathbf{x}, \mathbf{y}) = (\mathbf{x}^T\mathbf{y} + c)^d$
  - Finds curved boundaries and feature interactions
- **3. Radial Basis Function (RBF)** (The Default)
  - $K(\mathbf{x}, \mathbf{y}) = \exp(-\gamma||\mathbf{x} - \mathbf{y}||^2)$
  - "Local" similarity — points are similar only if close

Rule of Thumb: Start with **RBF** if you don't know the data structure.

SVM: Strengths and Weaknesses

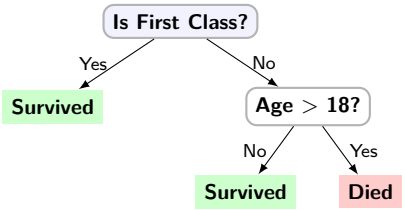
Advantages

- **High Dimensionality:** Works very well when features $\gg$ samples
- **Powerful:** Different kernels model complex non-linear relationships
- **Memory Efficient:** Only stores support vectors

Disadvantages

- **Noise Sensitivity:** Sensitive to outliers if C is large
- **Scalability:** Slow for large datasets ($> 10k$ samples)
- **Preprocessing:** Requires feature scaling
- **No Probabilities:** Does not natively output probabilities

Intuition: Split data by asking a sequence of Yes/No questions to separate classes.



Example: Titanic Survival

- **Goal:** Predict who survived
- **Root Question:** "Was the passenger in First Class?" (best separator)
- **Next Question:** If not, "Were they a child?"

The Core Question

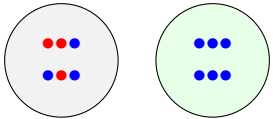
We have many features.

How do we decide which question to ask first?

How to Choose the Best Split? (Purity)

We want questions that lead to **"Pure" nodes** (containing only one class).

Visualizing Impurity



High Impurity (Mixed) | Zero Impurity (Pure)

Goal: Pick the split that maximizes the drop in impurity (Information Gain).

The Impurity Metrics:

1. **Gini Impurity (Default):**

$$G = 1 - \sum p_i^2$$

- $G = 0.5$: Max chaos (50/50)
- $G = 0.0$: Pure (one class)

2. **Entropy (Information):**

$$H = -\sum p_i \log_2(p_i)$$

What is p_i?
Proportion of samples belonging to class i in the current node.

Wait...Where is Gradient Descent?

Non-Tree Models

Method: Gradient Descent

- Loss surface is smooth (convex)
- Slide down the curve
- Update weights **w** iteratively

Decision Trees

Method: Greedy Search

- Try **every possible split** for every feature
- Pick the one with max Information Gain
- Repeat recursively until:
  - Node is **Pure** (100% one class)
  - Or hit a **Stop Condition** (e.g., Max Depth)

Decision Trees: Advantages and Disadvantages

Advantages

- **Simple:** Easy to understand
- **Interpretable:** "If X $>$ 5, then Y"
- **No Preprocessing:** No scaling or encoding needed

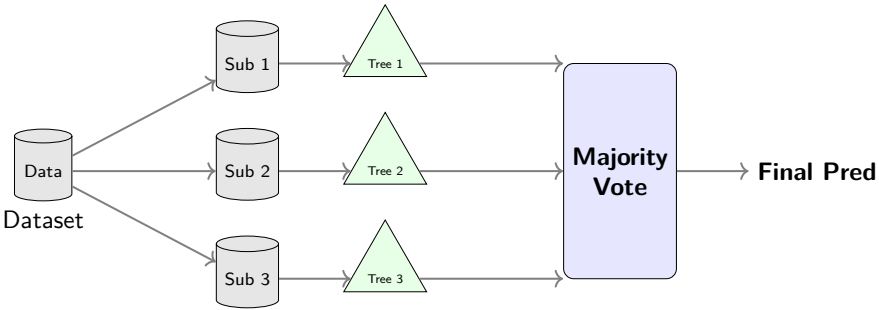
Disadvantages

- **High Variance:** Small data change = completely different tree
- **Overfitting:** Not setting stop conditions guarantees overfitting

Key hyperparameters: `max_depth`, `min_samples_split`, `min_samples_leaf`

Random Forest (Bagging)

Problem: Single trees are unstable (High Variance).
Solution: Train many trees on random subsets and average them (**Bagging**).



"Many uncorrelated weak models → One strong model."

Random Forest: Advantages and Disadvantages

Advantages

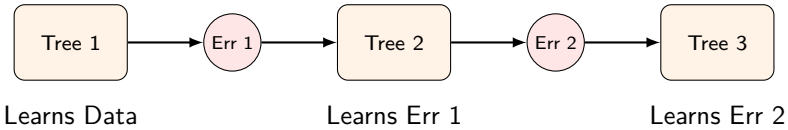
- **Robust:** Very hard to overfit
- **High Performance:** Excellent baseline
- **Low Tuning:** Works well with defaults
- **Feature Importance:** Tells you which features matter

Disadvantages

- **Slow Inference:** Must run through every tree
- **Memory Intensive:** Stores all trees
- **Less Interpretable:** Harder to explain than a single tree

Gradient Boosting: Correcting Mistakes

Problem: Random Forest just averages opinions. It doesn't learn from mistakes.
Solution: Train trees **sequentially**. Each tree fixes the errors of the previous one.



Common Implementations: **XGBoost** (best default), **LightGBM** (lightweight), **CatBoost** (minimal tuning)

Why is it called *Gradient* Boosting?

We are performing **Gradient Descent**, but in function space.

Standard Gradient Descent

Update **parameters** (w) to minimize loss:

$$\mathbf{w}_{new} = \mathbf{w}_{old} - \alpha \cdot \nabla \text{Loss}$$

Gradient Boosting

Update the **function** (F) by adding new trees:

$$F_{new}(x) = F_{old}(x) + \alpha \cdot \text{Tree}(x)$$

- With Squared Error Loss: $L = \frac{1}{2}(y - \hat{y})^2$
- The gradient is: $\nabla L = -(y - \hat{y})$
- The **Residual** $(y - \hat{y})$ is the negative gradient!

Gradient Boosting: Advantages and Disadvantages

Advantages

- **SOTA Performance:** Best accuracy for tabular data
- **Flexibility:** Custom loss functions
- **No Preprocessing:** Handles missing values natively

Disadvantages

- **Sensitive:** Harder to tune (learning rate, depth)
- **Overfitting:** Easy to overfit with too many trees
- **Serial:** Slow training (hard to parallelize)

The "No Free Lunch" Theorem

"Is there one Master Algorithm to rule them all?"

The Theorem (Wolpert, 1996)

Averaged over all possible data distributions, every classification algorithm has the same error rate on unseen data.

$$\mathbb{E}[\text{Error}(\text{Model}_A)] = \mathbb{E}[\text{Error}(\text{Model}_B)]$$

In Plain English:

- No model works best for *every* problem
- Great at images $\neq$ great at tabular data

The Implication

1. Make **assumptions** about your data ("it looks linear")
2. **Experiment** with multiple models from your toolbox
3. Use **cross-validation** (Day 4) to compare fairly

Day 5 Summary: The Complete ML Toolbox

Model	Type	Key Strength	Key Weakness
Linear / Logistic	Parametric	Interpretable, fast	Can't model non-linearity
k-NN	Non-Parametric	Simple, no training	Slow inference, curse of dim
SVM	Kernel	Powerful with kernels	Slow on large data
Decision Tree	Tree	Interpretable, no prep	High variance
Random Forest	Ensemble	Robust, good defaults	Memory heavy
Gradient Boosting	Ensemble	SOTA on tabular data	Hard to tune

Next: Neural Networks — from Perceptron to MLP (Day 6)

Credits

KAUST Academy

King Abdullah University of Science and Technology

KAUST–Mawhiba IOAI 2026 Training Program